Yihuai Zhang
School of Mechanical and Automotive Engineering,
South China University of Technology,
Guangzhou 510000, China
e-mail: zhangyh_scut@163.com

Baijun Shi[1]
Associate Professor
School of Mechanical and Automotive Engineering,
South China University of Technology,
Guangzhou 510000, China
e-mail: bjshi@scut.edu.cn

Xizhi Hu
Associate Professor
School of Mechanical and Automotive Engineering,
South China University of Technology,
Guangzhou 510000, China
e-mail: huxizhi@scut.edu.cn

Wandong Ai
School of Mechanical and Automotive Engineering,
South China University of Technology,
Guangzhou 510000, China
e-mail: tandou525@163.com

Low-Speed Vehicle Path-Tracking Algorithm Based on Model Predictive Control Using QPKWIK Solver

Automated valet parking is a part of autonomous vehicles. Path tracking is a vital capability of autonomous vehicles. In the scenario of automatic valet parking, the existing control algorithm will produce a high tracking error and a high computational burden. This paper proposes a path-tracking algorithm based on model predictive control to adapt to low-speed driving. By using the model predictive control method and vehicle kinematics model, a path tracking controller is designed. Combining the dual algorithm to further optimize the solver, a new quadratic programming (QP) knows what it knows (QPKWIK) solver is proposed. The simulation results show that the solution time of the QPKWIK solver is 25% less than that of the QP solver, and the tracking error is reduced by up to 41% compared with the QP solver. In the parking lot, the tracking performance is tested under four common scenarios, and the experimental results show that this controller has better tracking performance.
[DOI: 10.1115/1.4051645]

1 Introduction

Highly autonomous vehicles have attracted much interest in both industries and academia with the rapid development of technology [1]. Automatic valet parking is one of the application scenarios of autonomous driving. An automatic driving system must perform three separate tasks: perception [2], decision-making [3], and control [4]. As a vital technology in the automatic driving system, path tracking works together with the navigation system and the path planning system to control the lateral and longitudinal motion of the vehicle and guide the vehicle to travel along a predetermined path [5]. The predetermined path can be obtained by the path planning module. The path planning module is responsible for computing a safe, comfortable, and dynamically feasible path from the vehicle's current configuration to the goal configuration provided by the decision-making module [6]. Path planning in autonomous vehicles has been studied for years. Many authors divide the problem into global and local planning. Dijkstra algorithm is a graph searching algorithm that finds the single-source shortest path in the graph [7]. It was widely used in global path planning. A* algorithm is an extension of the Dijkstra algorithm. It enables a fast node search due to the implementation of heuristics [8]. The cost function in the A* algorithm defines the weights of the nodes. It is suitable for searching spaces mostly known a prior by the vehicle, however, it needs high computational cost. Rapidly exploring random tree allows fast planning in semistructured spaces, often used in local path planning [9]. Besides, the clothoid curves [10,11], polynomial curves [12], Bezier curves [13], and spline curves [14] were also applied in local path planning. The author in Ref. [12] proposed a novel path for lane change maneuvers based on a quantic function. The simulation proves that the lane change path generated by the polynomial curve is smooth and safe. The path planning module provides the desired path of the vehicle.

To track the predetermined path accurately and stably, the tracking error has become a key indicator to evaluate the tracking effect. Several kinds of tracking errors have been used in the path tracking algorithm. In simple geometric tracking controllers, such as pure pursuit (PP) [15], the steering angle is directly determined from the lateral deviation through the geometrical relationship between the vehicle and the desired path. But the main shortcoming of Pure Pursuit is in the selection of look-ahead distance. Another shortcoming of this algorithm is that it may not be able to negotiate discontinuous paths [16]. It is a common problem in geometric controllers. Improvement of this method has been proposed previously. An analytical method to tune the look-ahead distance base on a linearized model was proposed for path tracking control by Campbell [17]. Another geometric controller is "Stanley." It is an autonomous vehicle developed by Stanford University that won the second DARPA Grand Challenge in 2005. This method has been referenced and implemented in many studies. A nonlinear Stanley controller has been implemented on Hardware-in-the-loop testing and compares it against a linear method of double-loop control with feed-forward load disturbance compensation [18]. The results show that the Stanley controller was more superior with the better transient response and lower steady-state errors in curves. Moreover, unlike pure pursuit control, a look-ahead distance is not required for designing the controller. Linear quadratic regulator (LQR) is one of the most popular optimal controllers where the controller gain was determined using a linear quadratic optimization approach. Fan et al. [19] adopted LQR optimal control to achieve the closed-loop control of vehicle linear system to guarantee its stability and fast convergence property in the automatic parking systems. However, due to the absence of path feedback, the LQR controller performed badly, it undesirably provides steady-state errors when negotiating curvature paths [20].

Sliding mode control also has been successfully employed in many types of research on path tracking control for autonomous

[1]Corresponding author.

Contributed by the Dynamic Systems Division of ASME for publication in the Journal of Dynamic Systems, Measurement, and Control. Manuscript received March 27, 2021; final manuscript received June 23, 2021; published online September 13, 2021. Assoc. Editor: Mahdi Shahbahkti.

vehicles to ensure the robustness of the control system and to handle the nonlinear vehicle dynamics [21]. Wang et al. [22] proposed the saturation function in the control law to solve the Chattering issues, but the chattering phenomenon still exists. The author in Ref. [23] proposed a robust H_∞ output-feedback control strategy combined with mixed genetic algorithms/linear matrix inequality that was used to obtain proper static output-feedback gains. Besides, machine learning methods, with their capability to handle nonlinear systems, provide a new approach to deal with the nonlinear vehicle dynamic system problem. A control architecture combined traditional methods and deep reinforcement learning methods was proposed to correct control errors and reduce the time and effort of testing and tuning [24]. Shan et al. [25] proposed an adaptive Reinforcement learning-based weight-adjustment model to adjust the weight in PP_PID to tradeoff between the effects of PP and proportion integration differentiation (PID). This method can better deal with tracking errors and adapt to various tracking scenarios. Nevertheless, training a machine learning model adapted to real vehicles is still hard and needs high computational costs. Taking the S curve as the reference path, the tracking performance of each algorithm is shown in Fig. 1.

From above, a variety of control algorithms have been used in path tracking design, however, there are still many problems in balancing the robustness and tracking error. It would be of special interest to solve these problems for the path tracking control of autonomous vehicles. Model predictive control (MPC) is very suitable for dealing with vehicle stability constraints as well as changing vehicle and tire dynamics due to the ability to systematically including system constraints and future predictions [26–28]. Also, the inherent robustness of MPC guarantees the system robustness to some extent [29,30]. Constrained optimization problems can be solved by MPC in an optimal controller, which predicts the future state of the system and can handle state and control variables [31]. In practice, vehicle path-tracking by a model predictive controller is mathematically formulated in the design stage, but the system output is solved in real-time on a computing platform.

The MPC was generally implemented using a nonlinear vehicle model and linear vehicle model. Thus, the solution of MPC can be transformed into a nonlinear programming problem. This problem could be solved by the interior-point method [32] or the sequential quadratic programming (QP) method [33]. Real-time implementation of nonlinear MPC is limited owing to the computational complexity of the nonlinear programming problem. Dealing with this problem, linearization techniques can be used to replace the nonlinear programing problem, and finally converted it into a QP problem. Beginning with the unconstrained minimum of the objective function, the authors of Ref. [34] proposed an effective and numerically stable dual modus for positive definite QP. The authors of Ref. [35] proposed a simplified successive QP method that satisfactorily solves large-scale processing optimization problems with multiple variables, multiple constraints, and few degrees-of-freedom (DOF) at faster iteration speeds than other methods. For the issue of online calculation burden and complex tuning process, the online parameter selection method based on a look-up table is used to help the vehicle track the reference path under the constraints of stability and computing power. It realizes the online tuning of the controller and obtains excellent tracking performance [28]. In fact, MPC is greatly affected by computational complexity. Therefore, reducing the computational burden has become a key factor in the realization of the MPC method.

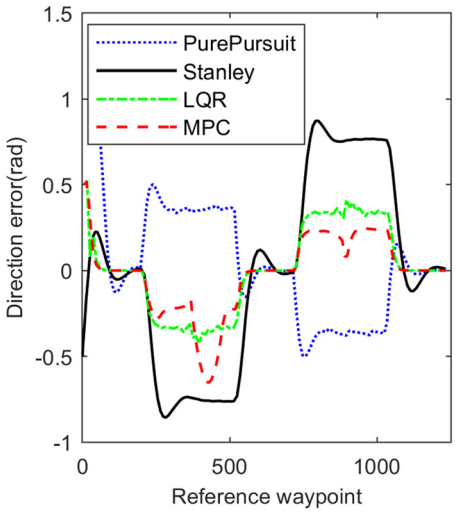

Fig. 1 Comparison of multiple algorithms tracking performance

Regardless of whether linear models or nonlinear models are used, most solutions in the MPC method are transformed into QP problems for further optimization. To further reduce computational burden and tracking error, this paper proposes a vehicle path-tracking algorithm based on model predictive control using the QP knows what it knows (QPKWIK) solver [34]. The contributions of this work are presented as follows:

- A kinematic model was established to satisfy the driving of the vehicle, and the vehicle motion state was described by a discrete state-space equation constructed through discretization processing, which obtained the input parameters of the path-tracking controller.
- Based on the vehicle motion equation, the vehicle constraints are added to the controller, the linear model is used to establish the model predictive controller, and the tracking error and heading error are used as the main influencing factors of the objective function.
- The model predictive controller solver is further optimized, and the QPKWIK solver is used to achieve a smaller computational cost and smaller tracking error. Experiments have proved that the vehicle tracking effect under this solver is good.

The remainder of this paper is organized as follows. Section 2 establishes the kinematics model and discrete state-space equation of the vehicle. Section 3 constructs the model predictive controller, and Sec. 4 optimizes the solver design and proposes the QPKWIK solver. Section 5 evaluates the tracking effect of the controller in an actual vehicle test. The paper concludes with Sec. 6.

2 Vehicle Kinematics Model

Vehicle kinematics defines the laws and constraints of vehicle motion from a geometric viewpoint, including the spatial position, driving speed, transient acceleration, and travel direction of the vehicle. These parameters will change over time. To simplify the vehicle motion process, the kinematics model regards the vehicle as a geometrically rigid body, ignoring the resultant force balance and torque balance in vehicle dynamics. The proposed tracking control is applied to well-smoothed roads and low driving speeds, which do not generally require vehicle handling dynamics. Therefore, this study establishes a vehicle kinematics model under driving control. The experimental subject was a pure electric car with rear-wheel drive and front-wheel steering.

2.1 Vehicle Kinematics Analysis. The main application scenarios of this research are parking lots and surrounding roads. To ensure driving safety, vehicles must travel at less than 5 km/h in a parking lot and less than 30 km/h on the roads outside the parking lot. Under the assumed conditions—a smooth driving road and a rigidly connected vehicle suspension system—the vertical displacement of the vehicle and the suspension coupling motion are ignored. The tire deformation in all directions, horizontal and vertical coupling, vehicle load changes and load transfer, vehicle

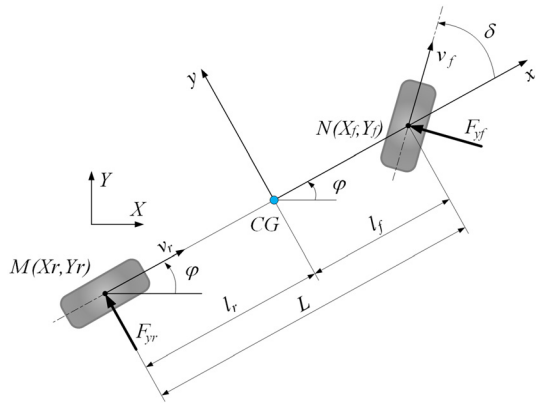

Fig. 2 Kinematics of the single-track model

horizontal and vertical aerodynamics, internal and external deviations of the steering wheel, and differential rotations of the front and rear wheels are also ignored. The vehicle kinematics of the bicycle model is based on the Ackerman steering principle under the above-mentioned assumptions [36].

The bicycle model of the vehicle is shown in Fig. 2. In the inertial coordinate system $O-XY$, (X_f, Y_f) and (X_r, Y_r) are the center coordinates of the front and rear axles of the vehicle, respectively, φ is the yaw angle of the vehicle body, v_r and v_f are the speeds of the center rear and front axles of the vehicle, respectively, L is the wheelbase of the vehicle. M and N denote the axes of the rear and front axles, respectively, δ is the equivalent front-wheel turning angle, and δ is calculated from the turning angles of the inner and outer steering wheels [37].

The 2DOF bicycle model has been used for designing the model predictive controller. The bicycle model has been built on a linear tire model [38]. The controller has been applied to the vehicle by using MATLAB/SIMULINK. The tire model is integrated into MATLAB/SIMULINK as one of the modules. Also, Pacejka magic formula tire model [39] and brush tire model [40] can be used in the simulation.

The steering angles are related as

$$\cot\alpha + \cot\beta = 2\cot\delta \tag{1}$$

where α and β denote the outer and inner wheel angles of the steering tire, respectively.

When the vehicle steering system satisfies the Ackerman steering principle, the vehicle steering-wheel angle ϕ_{steer} and the equivalent front-wheel angle δ are approximately linearly related as follows:

$$\phi_{\text{steer}} \approx \mathrm{i}_s \cdot \delta \tag{2}$$

where i_s is the equivalent steering gear ratio. From actual vehicle measurements, i_s was determined as 1.62.

Under the above assumptions, the speed at the center point of the rear axle of the vehicle is given by

$$v_r = \dot{X}_r\cos\varphi + \dot{Y}_r\sin\varphi \tag{3}$$

The kinematic constraints on the front and rear axles of the vehicle are, respectively, given by

$$\begin{cases} \dot{X}_f\sin(\varphi+\delta) - \dot{Y}_f\cos(\varphi+\delta) = 0 \\ \dot{X}_r\sin\varphi - \dot{Y}_r\cos\varphi = 0 \end{cases} \tag{4}$$

From Eqs. (3) and (4), we obtain

$$\begin{cases} \dot{X}_r = v_r\cos\varphi \\ \dot{Y}_r = v_r\sin\varphi \end{cases} \tag{5}$$

According to the geometric relationship between the front and rear axles of the vehicle, we have

$$\begin{cases} X_f = X_r + \mathrm{L}\cos\varphi \\ Y_f = Y_r + \mathrm{L}\sin\varphi \end{cases} \tag{6}$$

Substituting Eqs. (5) and (6) into Eq. (4), the vehicle yaw rate ω is obtained as

$$\omega = \dot{\varphi} = \frac{v_r\tan\delta}{L} \tag{7}$$

The steering radius and equivalent front-wheel angle are obtained from the vehicle yaw rate and vehicle speed, respectively,

$$\begin{cases} R = \dfrac{v_r}{\omega} \\ \delta = \arctan\left(\dfrac{L}{R}\right) \end{cases} \tag{8}$$

where R is the turning radius of the rear wheels.

From Eqs. (5) and (7), the vehicle kinematics model is obtained as

$$\begin{bmatrix} \dot{X}_r \\ \dot{Y}_r \\ \dot{\varphi} \end{bmatrix} = \begin{bmatrix} \cos\varphi \\ \sin\varphi \\ \tan\left(\dfrac{\delta}{L}\right) \end{bmatrix} v_r \tag{9}$$

Substituting Eq. (7) into Eq. (9), the vehicle kinematics model transforms as follows:

$$\begin{bmatrix} \dot{X}_r \\ \dot{Y}_r \\ \dot{\varphi} \end{bmatrix} = \begin{bmatrix} \cos\varphi \\ \sin\varphi \\ 0 \end{bmatrix} v_r + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \omega \tag{10}$$

2.2 Vehicle Corner Constraints. In vehicle kinematics analysis and model establishment, the main reference point is the midpoint of the rear axle of the vehicle. Owing to the geometric characteristics (length, width, and height) of the vehicle, the central movement of the rear axle cannot represent the movement of the entire vehicle. To understand the collision situation of the entire vehicle during the moving process, and impose safety constraints in the path planning and tracking control of the vehicle, the motion coverage space of the whole vehicle was analyzed in terms of the geometric relationship between the corner points of the vehicle body and the center of the rear axle. Ignoring the vehicles' height and simplifying its outline, the corner points of the vehicle body are geometrically related to the center of the vehicle's rear axle as shown in Fig. 3. The coordinates of points A, B, C, and D in Fig. 3 are given in Appendix. A.

2.3 Discretization of the Vehicle Motion State. According to Eq. (10), the vehicle kinematics model can be expressed as a state space equation

$$\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}, \mathbf{u}) \tag{11}$$

where $\boldsymbol{\xi} = [X, Y, \varphi]^{\mathrm{T}}$ and $\mathbf{u} = [v, \delta]^{\mathrm{T}}$.

The vehicle mechanism controls the lateral and longitudinal tracking movements of the vehicle by controlling the vehicle speed and wheel angle. The controlled tracking effect of the vehicle is fed back through the observed state quantity. Taking the

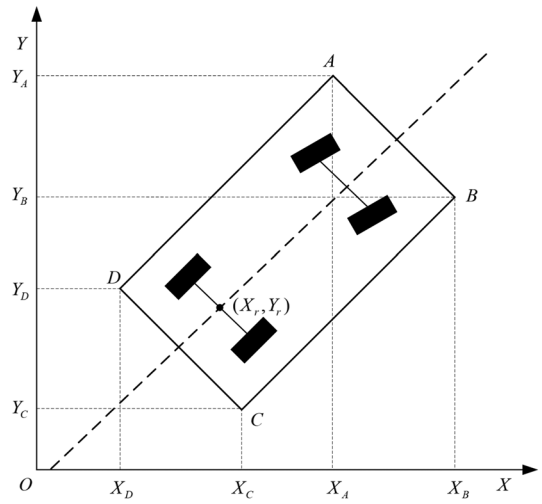

Fig. 3 Geometric relationship between the car-body corner points and rear-axle center (X_r, Y_r)

center of the rear axle of the vehicle as the reference point of vehicle motion, each point satisfies the reference path

$$\dot{\xi}_{\mathbf{r}} = f(\xi_{\mathbf{r}}, \mathbf{u}_{\mathbf{r}}) \tag{12}$$

where $\xi_{\mathbf{r}} = [X_r, Y_r, \varphi_r]^{\mathrm{T}}, \mathbf{u}_{\mathbf{r}} = [v_r, \delta_r]^{\mathrm{T}}$.

Expanding Eq. (12) as a Taylor series around the path reference point and ignoring the higher-order terms (Appendix B), turning it into a general form

$$\begin{aligned}\dot{\tilde{\xi}}_{\mathbf{r}} &= \begin{bmatrix} \dot{x} - \dot{x}_r \\ \dot{y} - \dot{y}_r \\ \dot{\varphi} - \dot{\varphi}_r \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & -v_r \sin\varphi_r \\ 0 & 0 & v_r \cos\varphi_r \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x - x_r \\ y - y_r \\ \varphi - \varphi_r \end{bmatrix} \\ &\quad + \begin{bmatrix} \cos\varphi_r & 0 \\ \sin\varphi_r & 0 \\ \dfrac{\tan\delta_r}{l} & \dfrac{v_r}{l\cos^2\delta_r} \end{bmatrix} \begin{bmatrix} v - v_r \\ \delta - \delta_r \end{bmatrix} \end{aligned} \tag{13}$$

Equation (13) is the linearized vehicle error model. For time-domain state prediction and real-time control of the vehicle, Eq. (13) is discretized as follows:

$$\tilde{\xi}(k+1) = \mathbf{A}_{k,t}\tilde{\xi}(k) + \mathbf{B}_{k,t}\tilde{\mathbf{u}}(k) \tag{14}$$

where

$$\mathbf{A}_{k,t} = \begin{bmatrix} 1 & 0 & -v_r \sin\varphi_r T \\ 0 & 1 & v_r \cos\varphi_r T \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{B}_{k,t} = \begin{bmatrix} \cos\varphi_r T & 0 \\ \sin\varphi_r T & 0 \\ \dfrac{\tan\varphi_r T}{l} & \dfrac{v_r T}{l\cos^2\delta_r} \end{bmatrix},$$

and T is the system sampling period.

3 Model Predictive Controller Design

Figure 4 schematizes the path-tracking controller used in our model. The dashed box in this figure is the main body of the path-tracking controller, which mainly includes the prediction model, system constraints, and objective function. The predictive model mathematically describes the path-tracking control system, and its construction is based on the vehicle model and state-space equations under the system constraints (the vehicle actuator constraints, vehicle driving-state constraints, and system-output control-volume smoothing constraints). For smooth tracking control, the objective function integrates the lateral and longitudinal control characteristics of the smart car and the calculation performance of the system platform. The MPC algorithm using QPKWIK solver is presented in Algorithm 1.

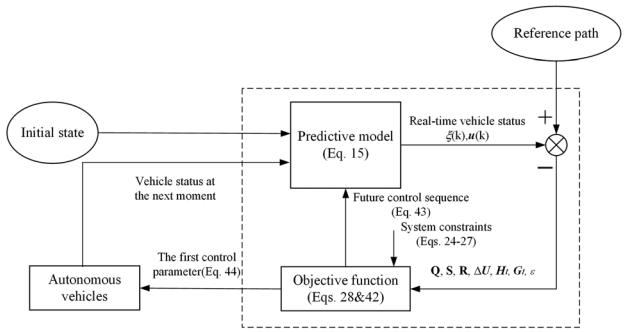

Fig. 4 Diagram of the path-tracking controller

Algorithm 1 MPC algorithm using QPKWIK solver

Input: Initial state, reference path, system constraints
Output: Control variable u(t)
1: **Step** 1 Calculate vehicle real state error and control error
2: **Step** 2 Update the state space equation $\xi(k)$
3: **Step 3** Calculate $\xi(k)\boldsymbol{U}, \boldsymbol{H}_t, \mathbf{G}_t, \boldsymbol{Q}, \boldsymbol{S}, \boldsymbol{R}, \varepsilon$
4: **Step 4** Minimize objective function using QPKWIK solver under system constraints and **Step 3** output
5: **Step 5** Generate the future control sequence and apply the first control parameter to vehicle
6: **Step 6** Update the next moment status to the predictive model
7: **if** the Vehicle reach to the end **then**
8: break
9: **else**
10: run **Step** 1 to **Step** 6
11 **end if**

3.1 Predictive Model. The vehicle motion state is linearized and discretized as follows:

$$\xi(k+1) = A_{k,t}\xi(k) + \mathbf{B}_{k,t}\boldsymbol{u}(k) \tag{15}$$

and $\psi(k|t)$ is given by

$$\boldsymbol{\psi}(k|t) = \begin{bmatrix} \xi(k|t) \\ u(k-1|t) \end{bmatrix} \tag{16}$$

The new state-space equation is then given by

$$\begin{cases} \boldsymbol{\psi}(k+1|t) = \tilde{\mathbf{A}}_{\boldsymbol{k},t}\boldsymbol{\psi}(k|t) + \tilde{\mathbf{B}}_{\boldsymbol{k},t}\Delta\mathbf{u}(k|t) \\ \boldsymbol{\eta}(k|t) = \tilde{\mathbf{C}}_{\boldsymbol{k},t}(k|t)\boldsymbol{\psi}(k|t) \end{cases} \tag{17}$$

where $\tilde{\mathbf{A}}_{\boldsymbol{k},t} = \begin{bmatrix} A_{k,t} & B_{k,t} \\ 0_{m\times n} & I_m \end{bmatrix}$, $\tilde{\mathbf{B}}_{\boldsymbol{k},t} = \begin{bmatrix} B_{k,t} \\ I_m \end{bmatrix}$, and $\tilde{\mathbf{C}}_{\boldsymbol{k},t} = \begin{bmatrix} C_{k,t} & 0 \end{bmatrix}$.

To further simplify the calculation, $\tilde{\mathbf{A}}_{\boldsymbol{k},t}$ and $\tilde{\mathbf{B}}_{\boldsymbol{k},t}$ are given as follows:

$$\begin{cases} \tilde{\mathbf{A}}_{\boldsymbol{k},t} = \tilde{\mathbf{A}}_t, & k = 1, \ldots, t+N-1 \\ \tilde{\mathbf{B}}_{\boldsymbol{k},t} = \tilde{\mathbf{B}}_t, & k = 1, \ldots, t+N-1 \end{cases} \tag{18}$$

Substituting Eq. (18) into Eq. (17) (Appendix C), we obtain

$$\boldsymbol{\psi}(k) = \tilde{\mathbf{A}}_t^{\boldsymbol{k}}\boldsymbol{\psi}(0) + \sum_{j=0}^{k-1}\left(\tilde{\mathbf{C}}_{\boldsymbol{k},t}\tilde{\mathbf{A}}_t^{\boldsymbol{k}-1-j}\tilde{\mathbf{B}}_t\Delta\mathbf{u}(j)\right) \tag{19}$$

$$\boldsymbol{\eta}(k) = \tilde{\mathbf{C}}_{k,t}\tilde{\mathbf{A}}_t^{k}\boldsymbol{\psi}(0) + \sum_{j=0}^{k-1}\left(\tilde{\mathbf{C}}_{k,t}\tilde{\mathbf{A}}_t^{k-1-j}\tilde{\mathbf{B}}_t\Delta\mathbf{u}(j)\right) \tag{20}$$

Setting the time domains of the system and control as N_p and N_c, respectively, the quantities of the system state and system control output in the prediction time domain are, respectively, obtained as follows:

$$\boldsymbol{\psi}(t+N_p|t) = \tilde{\mathbf{A}}_t^{N_p}\boldsymbol{\psi}(t|t) + \tilde{\mathbf{A}}_t^{N_p-1}\tilde{\mathbf{B}}_t\Delta\mathbf{u}(t|t) + \cdots + \tilde{\mathbf{A}}_t^{N_p-N_c-1}\tilde{\mathbf{B}}_t\Delta\mathbf{u}(t+N_c|t) \tag{21}$$

$$\boldsymbol{\eta}(t+N_p|t) = \tilde{\mathbf{C}}_t\tilde{\mathbf{A}}_t^{N_p}\boldsymbol{\psi}(t|t) + \tilde{\mathbf{C}}_t\tilde{\mathbf{A}}_t^{N_p-1}\tilde{\mathbf{B}}_t\Delta\mathbf{u}(t|t) + \cdots + \tilde{\mathbf{C}}_t\tilde{\mathbf{A}}_t^{N_p-N_c-1}\tilde{\mathbf{B}}_t\Delta\mathbf{u}(t+N_c|t) \tag{22}$$

Rearranging Eqs. (19) and (20), the control output of the system at future time t is expressed in matrix form as

$$\mathbf{Y}(t) = \boldsymbol{\Phi}_t\boldsymbol{\psi}(t|t) + \boldsymbol{\Theta}_t\Delta\mathbf{U}(t) \tag{23}$$

where

$$\mathbf{Y}(t) = \begin{bmatrix} \eta(t+1|t) \\ \eta(t+2|t) \\ \cdots \\ \eta(t+N_c|t) \\ \cdots \\ \eta(t+N_p|t) \end{bmatrix},\ \boldsymbol{\Phi}_t = \begin{bmatrix} \tilde{C}_t\tilde{A}_t^1 \\ \tilde{C}_t\tilde{A}_t^2 \\ \cdots \\ \tilde{C}_t\tilde{A}_t^{N_c} \\ \cdots \\ \tilde{C}_t\tilde{A}_t^{N_p} \end{bmatrix},\ \Delta\mathbf{U}(t) = \begin{bmatrix} \Delta u(t|t) \\ \Delta u(t+1|t) \\ \cdots \\ \Delta u(t+N_c|t) \end{bmatrix}$$

$$\text{and}\ \boldsymbol{\Theta}_t = \begin{bmatrix} \tilde{C}_t\tilde{B}_t & 0 & 0 & 0 \\ \tilde{C}_t\tilde{A}_t\tilde{B}_t & \tilde{C}_t\tilde{B}_t & 0 & 0 \\ \cdots & \cdots & \cdots & \cdots \\ \tilde{C}_t\tilde{A}_t^{N_c-1}\tilde{B}_t & \tilde{C}_t\tilde{A}_t^{N_c-2}\tilde{B}_t & \cdots & \tilde{C}_t\tilde{B}_t \\ \tilde{C}_t\tilde{A}_t^{N_c}\tilde{B}_t & \tilde{C}_t\tilde{A}_t^{N_c-1}\tilde{B}_t & \cdots & \tilde{C}_t\tilde{A}_t\tilde{B}_t \\ \cdots & \cdots & \cdots & \cdots \\ \tilde{C}_t\tilde{A}_t^{N_p-1}\tilde{B}_t & \tilde{C}_t\tilde{A}_t^{N_p-2}\tilde{B}_t & \cdots & \tilde{C}_t\tilde{A}_t^{N_p-N_c-1}\tilde{B}_t \end{bmatrix}$$

From Eq. (21), one observes that in the system prediction time domain, the system-state and system-control output can be calculated through $\boldsymbol{\psi}(k|t)$ and $\Delta\mathbf{U}(k)$, thus realizing a "system future-state prediction" function. Combining Eqs. (23) and (15), we can predict the motion and system states of the vehicle in the time domain. The vehicle control is optimized by optimizing the solution of $\Delta\mathbf{U}(k)$ in the control time domain.

3.2 System Constraints. The optimization problem of the path-following controller is a type of pan-domain problem, meaning that the range of globally optimal solutions may exceed the actual application requirements. The reference path must be tracked under various constraints: the driving constraints, system execution constraints, acceleration and deceleration constraints, and vehicle-driving stability constraints. Following the vehicle model and vehicle travel restrictions, the designed path-tracking controller constrains the front-wheel angle and its increment, the acceleration of the vehicle, and the corner driving of the vehicle.

The front-wheel angle output by the path-tracking controller controls the driving direction of the vehicle, but the vehicle is limited by the steering system, and the left and right steering capabilities are limited. In a tracking experiment of the vehicle steering system, the front-wheel angle was constrained to the following range:

$$-37\,\text{deg} \le \delta_f \le 37\,\text{deg} \tag{24}$$

To ensure the stability and safety of the driving vehicle, excessive steering must be prevented by constraining the increment of the front-wheel angle. Due to the differences in the structure of the steering system of different vehicles, the front wheel angle increment needs to be measured through experiments. Through experiments, the front-wheel angle increment on a flat road is limited to

$$-0.5\,\text{deg} \le \Delta\delta_f \le 0.5\,\text{deg} \tag{25}$$

Since the vehicle is running in a low-speed environment, to prevent the tracking performance from being affected by the instantaneous increase or decrease of the vehicle speed, and at the same time, considering the comfort of passengers, the acceleration range is determined. Through experiments, the vehicle tracking performance and passenger comfort are the best in the following acceleration ranges:

$$-0.06\,\text{m/s}^2 \le a \le 0.06\,\text{m/s}^2 \tag{26}$$

To ensure that the vehicle does not collide with the edge of the road during driving, the minimum distance between the corner point of the vehicle and the edge of the road is set. The vehicle is subject to the following constraints:

$$\|(X_{i,k+1}, Y_{i,k+1})\| \le \|(X_{b,k+1}, Y_{b,k+1})\| - D_s \tag{27}$$

where $(X_{i,k+1}, Y_{i,k+1})$ is the coordinate of the corner i of the vehicle at the time $k+1$, $(X_{b,k+1}, Y_{b,k+1})$ is the road boundary of the vehicle's position at the time $k+1$, and D_s is the anticollision safety distances. Considering obstacle avoidance and sensor accuracy, D_s is set at 30 cm after the experiment.

3.3 Objective Function. As the control variable of the path-following controller is unknown, we must set a regular objective function. The control variables include the heading angle and the speed of the vehicle. Based on the optimal solution, a sequence of control variables is obtained in the system-control time domain. The objective function must ensure fast and stable tracking of the vehicle along the reference path. Accounting for the vehicle's tracking deviation and optimal control of the control quantity, we obtain

$$\begin{cases} J(\boldsymbol{\psi}(t),\Delta\mathbf{U}(t)) = \sum_{i=1}^{Np}\|\boldsymbol{\eta}(t+i|t) - \boldsymbol{\eta}_{ref}(t+i|t)\|_Q^2 + \sum_{i=1}^{Np}\|\mathbf{u}(\mathbf{t}+\mathbf{i}|\mathbf{t})\|_S^2 \\ \qquad + \sum_{i=1}^{Nc-1}\|\Delta\mathbf{u}(t+i|t)\|_R^2 \\ \left(\begin{array}{c} \Delta\mathbf{U}_t = [\Delta u_t, \Delta u_{t+1}, \ldots, \Delta u_{t+N_c-1}]^{\mathrm{T}} \\ \Delta u(t) = u(t) - u(t-1) \end{array}\right) \end{cases} \tag{28}$$

where $\sum_{i=1}^{Np}\|\boldsymbol{\eta}(t+i|t) - \boldsymbol{\eta}_{ref}(t+i|t)\|_Q^2$ is the degree of deviation between the actual output of system η and the reference output η_{ref}, $\sum_{i=1}^{Np}\|\mathbf{u}(t+i|t)\|_S^2$ is the system state and $\sum_{i=1}^{Nc-1}\|\Delta\mathbf{u}(t+i|t)\|_R^2$ is the system-control increment. The first, second and third of these terms reflect the vehicle's ability to track the reference path, the predictive ability of the system, and the smoothness of the control system, respectively. ΔU is the control sequence, N_p and N_c are the control and prediction time domains, respectively, and Q, S, R define the controller weight matrices. Since the length

of the horizon affects the computational burden, the length of the horizon is too long, resulting in a high computational burden [41]. In this article, we set $Np = 60$ and $Nc = 30$. Under this condition, the tracking error of the vehicle is the smallest and the computational burden is not high.

The system-control sequence in the future control time domain is obtained by combining Eq. (24). As a real-time tracking system, the vehicle path-tracking controller may not admit an optimal solution. In practical applications, control lag and system-solution collapse are prevented by adding a relaxation factor [42]. The objective function is expressed as follows:

$$\begin{cases} J(\boldsymbol{\psi}(t),\boldsymbol{\Delta U}(t)) = \sum_{i=1}^{Np}\|\boldsymbol{\eta}(t+i|t)-\boldsymbol{\eta}_{ref}(t+i|t)\|_Q^2+\sum_{i=1}^{Np}\|\mathbf{u}(\mathbf{t}+\mathbf{i}|\mathbf{t})\|_S^2 \\ \qquad +\sum_{i=1}^{Nc-1}\|\boldsymbol{\Delta u}(t+i|t)\|_R^2+\rho\varepsilon^2 \\ \begin{pmatrix} \boldsymbol{\Delta U}_t=[\Delta u_t,\Delta u_{t+1},\ldots,\Delta u_{t+N_c-1}]^{\mathrm{T}} \\ \Delta u(t)=u(t)-u(t-1) \end{pmatrix} \end{cases} \tag{29}$$

where ε is the relaxation factor, ρ is the weight coefficient.

4 Solver Design Optimization

4.1 Quadratic Programming Solver Design. The path-tracking controller based on MPC computes the output sequence of the system control, which determines the driving action of the vehicle at the next moment. This problem belongs to the class of optimization problems with constraints. Therefore, the objective function Eq. (29) is converted to a QP optimization problem with linear or nonlinear constraints. By Eq. (29), we obtain

$$\begin{aligned} J(t) &\leftarrow \varsigma_{\boldsymbol{\eta}}(t)+\varsigma_{\boldsymbol{u}}(t)+\varsigma_{\boldsymbol{\Delta u}}(t) \\ \varsigma_{\boldsymbol{\eta}}(t) &\leftarrow \sum^{Np}\|\boldsymbol{\eta}(\mathbf{t})-\boldsymbol{\eta}_{\mathbf{ref}}(\mathbf{t})\|_Q^2 \\ \varsigma_{\boldsymbol{u}}(t) &\leftarrow \sum^{Np}\|\mathbf{u}(\mathbf{t})\|_S^2 \\ \varsigma_{\boldsymbol{\Delta u}}(t) &\leftarrow \sum^{Nc-1}\|\boldsymbol{\Delta u}(\mathbf{t})\|_R^2 \end{aligned} \tag{30}$$

for $\varsigma_{\boldsymbol{\eta}} \leftarrow \sum^{N_p}\|\boldsymbol{\eta}(t)-\boldsymbol{\eta}_{\mathrm{ref}}(t)\|_Q^2$. We also define the matrices $\boldsymbol{\Gamma}$, $\boldsymbol{\Gamma}_{\mathrm{ref}}$, and $\boldsymbol{\Delta U}$ as follows:

$$3pt\boldsymbol{\Gamma} = \begin{bmatrix} \eta(1) \\ \eta(1) \\ \cdots \\ \eta(k) \\ \cdots \\ \eta(N) \end{bmatrix}, \quad \boldsymbol{\Gamma}_{\mathrm{ref}} = \begin{bmatrix} \eta_{ref}(1) \\ \eta_{ref}(1) \\ \cdots \\ \eta_{ref}(k) \\ \cdots \\ \eta_{ref}(N) \end{bmatrix}, \quad \boldsymbol{\Delta U} = \begin{bmatrix} \Delta u(1) \\ \Delta u(1) \\ \cdots \\ \Delta u(k) \\ \cdots \\ \Delta u(N) \end{bmatrix} \tag{31}$$

Substituting Eqs. (23) and (31) into $\varsigma_{\eta}(t)$, we get

$$\varsigma_{\boldsymbol{\eta}}(t) = \|\boldsymbol{\Theta\Delta U}-\mathbf{B}\|_Q^2, \mathbf{B} = \boldsymbol{\Gamma}_{\mathrm{ref}}-\boldsymbol{\Phi\psi}(0) \tag{32}$$

Expanding Eq. (32) gives

$$\begin{aligned} \varsigma_{\boldsymbol{\eta}} &= \|\boldsymbol{\Theta\Delta U}-\mathbf{B}\|_Q^2 \\ &= (\boldsymbol{\Theta\Delta U}-\mathbf{B})^{\mathrm{T}}\mathbf{Q}(\boldsymbol{\Theta\Delta U}-\mathbf{B}) \\ &= \boldsymbol{\Delta U}^{\mathbf{T}}\boldsymbol{\Theta}^{\mathbf{T}}\mathbf{Q}\boldsymbol{\Theta\Delta U}-2(\boldsymbol{\Theta}^{\mathbf{T}}\mathbf{QB})^{\mathrm{T}}\boldsymbol{\Delta U}+\mathbf{B}^{\mathbf{T}}\mathbf{QB} \end{aligned} \tag{33}$$

The term $\mathbf{B}^{\mathbf{T}}\mathbf{QB}$ does not affect the optimal solution, so is ignored.

For $\varsigma_{\boldsymbol{u}}(t) \leftarrow \sum^{Np}\|\mathbf{u}(t)\|_S^2$, we set matrices K and M as follows:

$$\mathbf{K} = \begin{bmatrix} 1 & 0 & 0 & \ldots & 0 \\ 1 & 1 & 0 & \ldots & 0 \\ 1 & 1 & 1 & \ldots & 0 \\ \ldots & \ldots & \ldots & & \ldots \\ 1 & 1 & 1 & \ldots & 1 \end{bmatrix}, \mathbf{M} = \mathbf{K}\otimes\mathbf{I} \tag{34}$$

Hence, we have

$$\mathbf{U} = \mathbf{M\Delta U}+\mathbf{U}_{-1} \tag{35}$$

where $\mathbf{U}$, $\boldsymbol{\Delta U}$, $\mathbf{U}_{-1}$ are defined as: $\mathbf{U} = \begin{bmatrix} u_1 \\ u_1 \\ \ldots \\ u_N \end{bmatrix}$, $\boldsymbol{\Delta U} = \begin{bmatrix} \Delta u_1 \\ \Delta u_1 \\ \ldots \\ \Delta u_N \end{bmatrix}$, $\boldsymbol{U}_{-1} = \begin{bmatrix} u_{-1} \\ u_{-1} \\ \ldots \\ u_{-1} \end{bmatrix}$.

Substituting Eq. (35) into $\varsigma_u(t)$, we obtain

$$\begin{aligned} \varsigma_{\boldsymbol{u}} &= \|U\|_S^2 = (\mathbf{M\Delta U}+\mathbf{U}_{-1})^{\mathrm{T}}\mathbf{S}(\mathbf{M\Delta U}+\mathbf{U}_{-1}) \\ &= \boldsymbol{\Delta U}^{\mathrm{T}}(\mathbf{M}^{\mathrm{T}}\mathbf{SM})\boldsymbol{\Delta U}+2(\mathbf{U}_{-1}^{\mathrm{T}}\mathbf{SM})\boldsymbol{\Delta U}+\mathbf{U}_{-1}^{\mathrm{T}}\mathbf{SU}_{-1} \end{aligned} \tag{36}$$

For $\varsigma_{\boldsymbol{\Delta u}}(t) \leftarrow \sum^{Nc-1}\|\boldsymbol{\Delta u}(t)\|_R^2$ and using $\Delta u(t)$, we obtain

$$\varsigma_{\boldsymbol{\Delta u}} = \boldsymbol{\Delta U}^{\mathbf{T}}\mathbf{R}\boldsymbol{\Delta U} \tag{37}$$

Combining Eq. (30), the objective function is obtained as

$$\begin{aligned} \min_{\boldsymbol{\Delta u}}\mathbf{J} &= \varsigma_{\boldsymbol{\eta}}+\varsigma_{\mathbf{u}}+\varsigma_{\boldsymbol{\Delta u}} = \boldsymbol{\Delta U}^{\mathbf{T}}\boldsymbol{\Theta}^{\mathbf{T}}\mathbf{Q}\boldsymbol{\Theta\Delta U}-2\left(\boldsymbol{\Theta}^{\mathbf{T}}\mathbf{QB}\right)^{\mathbf{T}}\boldsymbol{\Delta U}+\mathbf{B}^{\mathbf{T}}\mathbf{QB} \\ &+\boldsymbol{\Delta U}^{\mathrm{T}}(\mathbf{M}^{\mathrm{T}}\mathbf{SM})\boldsymbol{\Delta U}+2(\mathbf{U}_{-1}^{\mathrm{T}}\mathbf{SM})\boldsymbol{\Delta U}+\mathbf{U}_{-1}^{\mathrm{T}}\mathbf{SU}_{-1}+\boldsymbol{\Delta U}^{\mathbf{T}}\mathbf{R}\boldsymbol{\Delta U} \end{aligned} \tag{38}$$

which simplifies to

$$\begin{aligned} \min_{\boldsymbol{\Delta u}}\mathbf{J} &= \boldsymbol{\Delta U}^{\mathbf{T}}\boldsymbol{\Theta}^{\mathbf{T}}\mathbf{Q}\boldsymbol{\Theta\Delta U}-2\left(\boldsymbol{\Theta}^{\mathbf{T}}\mathbf{QB}\right)^{\mathbf{T}}\boldsymbol{\Delta U}+\boldsymbol{\Delta U}^{\mathrm{T}}(\mathbf{M}^{\mathrm{T}}\mathbf{SM})\boldsymbol{\Delta U} \\ &+2\left(\mathbf{U}_{-1}^{\mathrm{T}}\mathbf{SM}\right)\boldsymbol{\Delta U}+\boldsymbol{\Delta U}^{\mathrm{T}}\mathbf{R}\boldsymbol{\Delta U}+\rho\varepsilon^2 \\ &= \boldsymbol{\Delta U}^{\mathrm{T}}\mathbf{H}\boldsymbol{\Delta U}+\mathbf{G}^{\mathrm{T}}\boldsymbol{\Delta U}+\rho\varepsilon^2 \end{aligned} \tag{39}$$

with $\mathbf{H} = \boldsymbol{\Theta}^{\mathbf{T}}\mathbf{Q}\boldsymbol{\Theta}+\mathbf{M}^{\mathbf{T}}\mathbf{SM}+\mathbf{R}$, $\mathbf{G} = -\boldsymbol{\Theta}^{\mathrm{T}}\mathbf{QB}+(\mathbf{U}_{-1}^{\mathbf{T}}\mathbf{SM})^{\mathrm{T}}$.

Accounting for the relaxation factor, we, respectively, set $\mathbf{H_t}$ and $\mathbf{G_t}$ as

$$\mathbf{H_t} = \begin{bmatrix} \mathbf{H} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \end{bmatrix}, \quad \mathbf{G_t} = \begin{bmatrix} \mathbf{G} \\ \mathbf{0} \end{bmatrix} \tag{40}$$

The objective function is then expressed as

$$\mathbf{J}(\boldsymbol{\psi}(t),\boldsymbol{\Delta U}(t)) = [\boldsymbol{\Delta U}(\mathbf{t})^{\mathrm{T}},\varepsilon]^{\mathrm{T}}\mathbf{H}_t[\boldsymbol{\Delta U}(t)^{\mathrm{T}},\varepsilon]+\mathbf{G}_t[\boldsymbol{\Delta U}(\mathbf{t})^{\mathrm{T}},\varepsilon]+P_t \tag{41}$$

where P_t is a system constant.

To solve the constrained optimization problem, the path-following controller based on MPC solves the following QP optimization problem:

$$\begin{aligned} &\min_{\Delta U,\varepsilon}\left[\boldsymbol{\Delta U}(t)^{\mathrm{T}},\varepsilon\right]^{\mathrm{T}}\mathbf{H}_t\left[\boldsymbol{\Delta U}(t)^{\mathrm{T}},\varepsilon\right]+\mathbf{G}_t\left[\boldsymbol{\Delta U}(t)^{\mathrm{T}},\varepsilon\right] \\ &\text{subject to}\cdot\Delta U_{\min}\le\boldsymbol{\Delta U}(k)\le\Delta U_{\max}, k=t,\ldots,t+H_c-1 \\ &\cdot U_{\min}\le u(t-1)+\sum_{i=t}^{k}\boldsymbol{\Delta U}(i)\le U_{\max}, \\ &\cdot k=t,\ldots,t+H_c-1, \\ &\cdot Y_{\min}-\varepsilon\le\boldsymbol{\Phi}_t\boldsymbol{\psi}(t|t)+\boldsymbol{\Theta}_t\boldsymbol{\Delta U}(t)\le Y_{\max}+\varepsilon, \\ &\cdot\|\boldsymbol{\psi}_{i,k+1}\|\le\|\boldsymbol{\psi}_{d,k+1}\|-D_s, \\ &\cdot\varepsilon>0 \end{aligned} \tag{42}$$

The incremental control sequence is then obtained as follows:

$$\Delta \mathbf{U}_t^* = [\Delta u_t^*, \Delta u_{t+1}^*, \ldots, \Delta u_{t+N_C-1}^*]^T \tag{43}$$

The first element of the control quantity sequence output by the control is

$$u(t) = u(t-1) + \Delta u_t^* \tag{44}$$

The tracked reference path is controlled by iterating the above optimization solution in the system time domain. The control output of the designed path-tracking controller involves the front-wheel angle and driving speed. To track the reference path, the steering, acceleration, and deceleration of the vehicle are completed through a vehicle control unit.

Based on the above-developed objective function, the QP problem is formulated as

$$\min_x \frac{1}{2} x^T H x + f^T x \quad \text{subject to} \begin{cases} A \cdot x \le b \\ Aeq \cdot x = beq \\ lb \le x \le ub \end{cases} \tag{45}$$

which has a global minimum if H is a positive semidefinite matrix and the feasible region of the constraints is nonempty. If the feasible region contains the lower bound of the objective function, the optimization problem has a global minimum; moreover, if H is a positive definite matrix, the global minimum is unique.

Combining Eqs. (42) and (45), the objective function that optimizes the solution is given as follows [43]:

$$[X, Fval, Exitflag] = \text{QUADPROG}(H, f, A, b, Aeq, beq, lb, ub, X_0, \text{Options}) \tag{46}$$

where $(H, f, A, b, Aeq, beq, lb, ub, X_0, \text{Options})$ and $[X, Fval, Exitflag]$ are the input and output of the objective function, respectively, and QUADPROG() is the function body. The input variables are the Hessian matrix H, the gradient matrix f, A, and b satisfying the linear constraint $A \cdot x \le b$ (if the constraint does not exist, then $A = 0, b = 0$), Aeq and beq satisfying the linear constraint, the upper and lower limits of the optimal solution of the objective function (lb and ub, respectively), the starting point X_0 of the iterative objective-function solution, and the solution method Options. The output variables are the optimal solution X of the objective function under all system constraints, the function value $Fval$ of the objective function at X, and the solution termination flag Exitflag.

4.2 Quadratic Programming Knows What It Knows Solver Optimization. Reducing the computational burden and reducing tracking errors are the main ways to improve MPC performance. Many studies transform solutions in MPC into QP problems and use QP solvers to solve them [44]. Since MPC needs to perform real-time rolling optimization, the QP solver will also cause a high computational burden, which is of great significance to the optimization of the solver. To solve this subproblem effectively and reduce the computational burden, we proposed a new quadratic programming solver, QPKWIK, based on the successive quadratic programming problem [45] and dual algorithm [34]. When applying it to the MPC algorithm, the original Hessian matrix can be updated using the inverse Cholesky factor of the Hessian matrix in each iteration.

The QPKWIK solver performs a Cholesky decomposition of the Hessian matrix to obtain L, then inverts the coefficient matrix to obtain L_{inv}, and uses the inverted coefficient matrix L_{inv} to update the iterative objective function for the optimal-solution calculation. Combining the steps of the QPKWIK solver and Cholesky matrix decomposition, we have

$$[L, p] = \text{chol}(H, \text{'lower'}) \; p = \begin{cases} 0, & H \leftarrow \text{Positive Definite} \\ N_+, & H \leftarrow \text{Otherwise} \end{cases}$$
$$L_{inv} = \text{inv}(L) \tag{47}$$

where chol() stands for Cholesky matrix factorization [46] and inv () denotes a matrix inversion. The QPKWIK algorithm is defined as Algorithm 2.

Algorithm 2 QPKWIK algorithm

Input: Objective function, all system constraints
Output: Optimal solution for future control sequence
1: **Step** 1 Perform Choelesky decomposition of the Hessian matrix and find the inverse matrix through chol() and inv()
2: **Step** 2 Use Karush-Kuhn-Tucker (KKT) conditions to solve unconstrained solutions
3: **Step 3** Add one constrain to the solution
4: **Step 4** Determine the search direction in the dual space
5: **Step 5** Compute the step length in dual space and take step optimization
6: **if** Solution satisfies all the constraints **then**
7: break
8: **else**
9: run **Step** 1 to **Step** 6
10 **end if**

The QPKWIK solver improves the efficiency of effective set identification and solves the search direction problem of an infeasible quadratic programming subproblem by relaxing the constraint of the variable without violating the boundary of the variable. In this way, it improves the flexibility of the iterative calculation.

4.3 Tracking Effect Comparison. To verify the influences of the designed QPKWIK and QP solvers on the solution of the MPC-based algorithm, a simulation environment was built on the MATLAB/SIMULINK software platform. The tracking-control effects of the two solvers were investigated in this environment. The tracking ability of the MPC algorithm was tested in the simulation model, and two curved paths (one circular, the other gentle) were set as reference paths by the path generation function. To set the reference paths, the algorithm calls the QUADPROG() or QPKWIK() function and obtains the driving trajectory of the vehicle (see Fig. 5).

The QUADPROG and QPKWIK results of the tracking simulation are compared in Table 1. In terms of both tracking effect and iterative calculation speed, the solver using the QUADPROG() function outperformed the solver using the QPKWIK() function, confirming the advantages of the proposed QPKWIK solution.

In the simulation experiments, the MPC algorithm based on the QPKWIK solver achieved superior path-tracking control. The summed tracking errors of the circular and gentle curves in the QPKWIK result were 25.63% and 41.79% smaller, respectively, than those in the QP result, and the average single-cycle solution times of the QPKWIK solver were reduced by 26.03% and 24.77%, respectively, from those of the QP solver.

5 Experimental Results and Discussion

Sections 3 and 4 presented our path-tracking control algorithm based on MPC and verified the designed QPKWIK solver in a tracking-control simulation. Owing to the complete nonlinearity of the vehicle and the simplified iterative processing of the path-tracking control, some system parameters cannot be accurately obtained, and the effectiveness of the algorithm cannot be determined in real environments. Therefore, the vehicle platform was modified and combined with a vehicle model and a high-precision

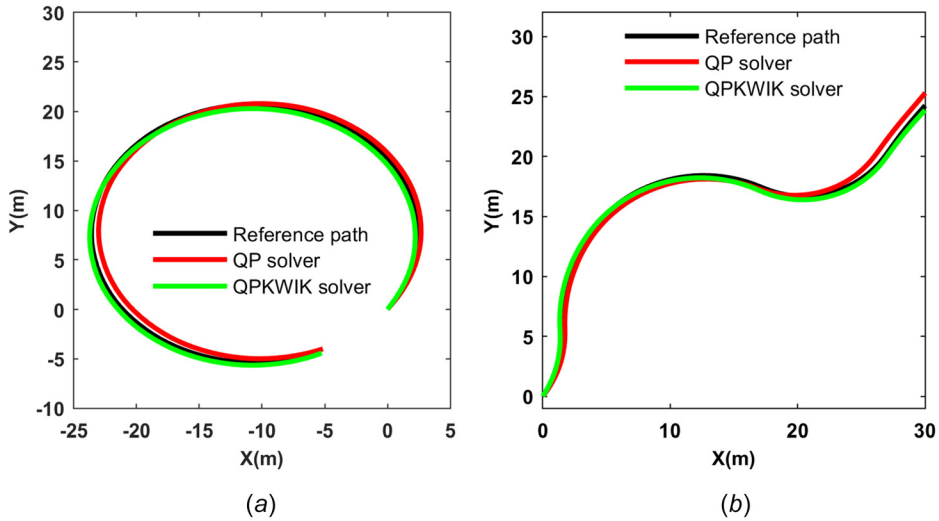

Fig. 5 Comparison of tracking results along the reference paths: (*a*) circular curve and (*b*) gradual curve

Table 1 Comparative analysis of the tracking simulation results

		QPKWIK	QP	Difference
Sum of tracking error (m)	Circular curve	34.03	45.76	+25.63%
	Gentle curve	21.90	37.62	+41.79%
Average single cycle solution time (ms)	Circular curve	75.35	101.87	+26.03%
	Gentle curve	77.81	103.43	+24.77%

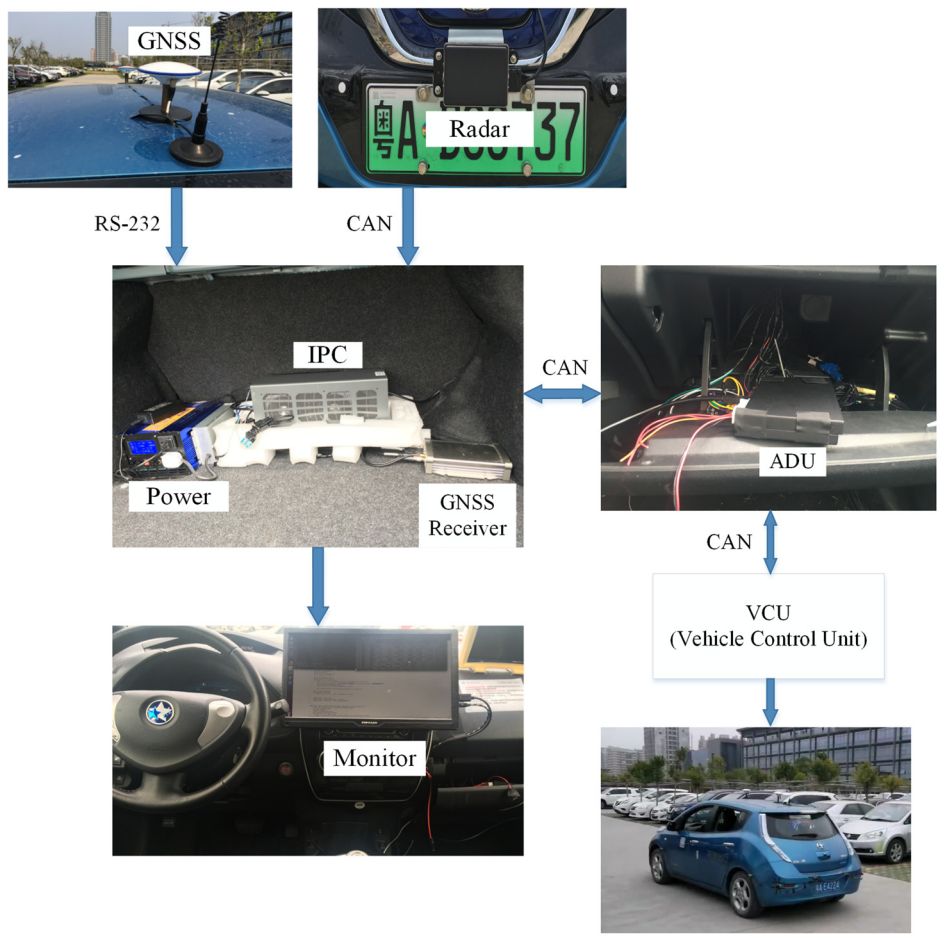

Fig. 6 Control system of the experimental vehicle

Table 2 The RMSE of position and direction

Driving condition	Error type	RMSE	Normalized RMSE
Straight driving	Position	38.8397 cm	63.03%
	Direction	0.0393 rad	56.86%
Right-angle turning	Position	28.2931 cm	35.67%
	Direction	0.0792 rad	16.75%
Rectangular driving	Position	49.2486 cm	43.88%
	Direction	1.4453 rad	23.01%
Obstacle avoidance driving	Position	41.0903 cm	41.42%
	Direction	0.1223 rad	26.41%

electronic map. The control effect of the designed path-tracking algorithm was then verified in a parking lot. The control system of the experimental vehicle is shown in Fig. 6.

To verify the path-tracking performance of the experimental vehicle system, control driving experiments were performed in a real vehicle under several working conditions. The experimental environment was an empty-field parking lot, so the maximum speed limit was set to 5 km/h from a safety perspective. The experiment uses the produced high-precision map as a driving navigation reference and verifies the path-tracking effect along the reference path generated by the path planning algorithm.

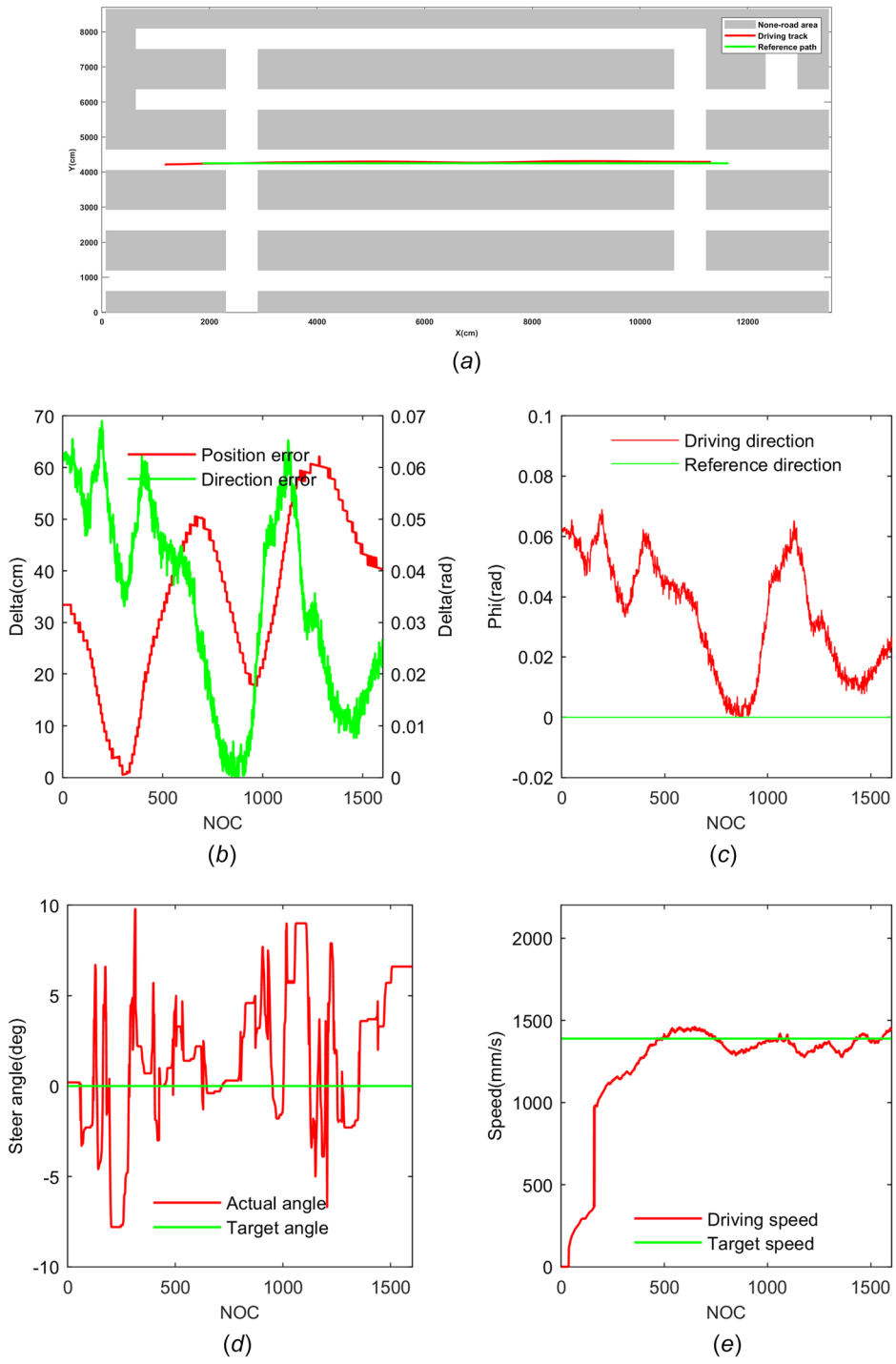

Fig. 7 Straight-line driving condition: (*a*) tracking-control effect chart, (*b*) vehicle position and heading error, (*c*) vehicle driving and reference directions, (*d*) vehicle steering wheel target and execution angles, and (*e*) vehicle target and driving speeds (NOC = number of cycles)

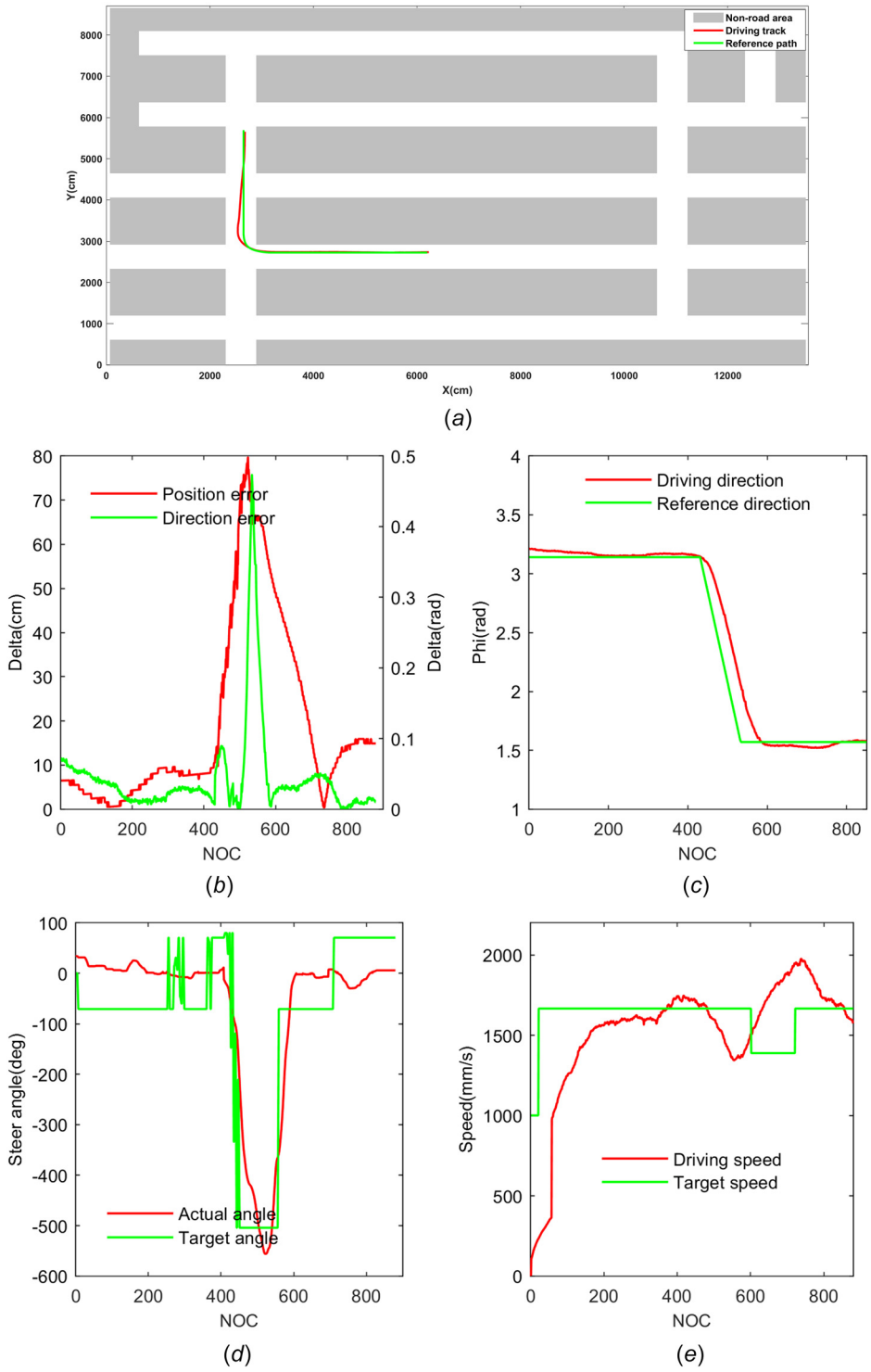

Fig. 8 Driving at right angles: (*a*) tracking-control effect chart, (*b*) vehicle position and heading error, (*c*) vehicle driving and reference directions, (*d*) vehicle steering wheel target and execution angles, and (*e*) vehicle target and driving speeds

Table 2 shows the root-mean-square error (RMSE) of position and direction in different conditions.

5.1 Straight Driving Condition. First, the driving conditions were tested along a straight line of road in the parking lot. The path-tracking results are shown in Fig. 7. Figure 7 shows the tracking performance of the vehicle along the straight section of the road. Figure 7(*a*) maps the straight route through the parking lot. In Fig. 7(*b*), the left vertical axis represents the parallel difference between the actual position and the reference path, and the right vertical axis represents the difference between the actual heading and the reference heading. These differences, obtained during the vehicle-driving process, respectively, reflect the deviations in vehicle-tracking control distance and the approachability of the vehicle. The maximum positional and heading deviations were 37.3 cm and 0.069 rad, respectively. The RMSE of position and direction were 38.8397 cm and 0.393 rad, respectively. The

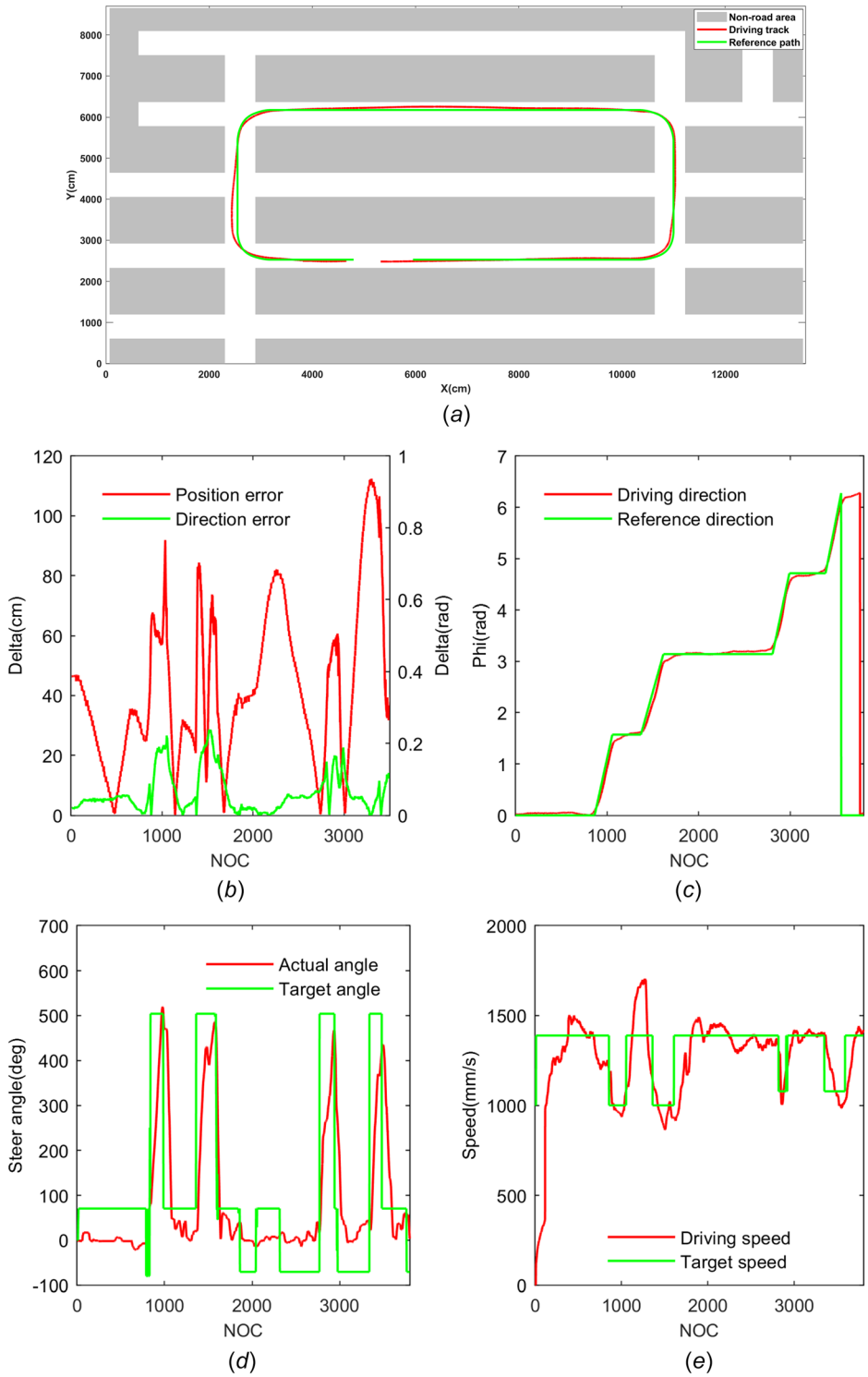

Fig. 9 Rectangular driving: (*a*) tracking-control effect chart, (*b*) vehicle position and heading error, (*c*) vehicle driving and reference directions, (*d*) vehicle steering wheel target and execution angles, and (*e*) vehicle target and driving speeds

deviations were most obvious along two sections of road: NOC = [300, 400] and NOC = [980, 1150] (where NOC = number of cycles). The vehicle heading changed smoothly during the tracking process (Fig. 7(*c*)), and the steering-wheel angle fluctuated between [−10 deg, 10 deg] (Fig. 7(*d*)). The entire control was stable and smooth. The vehicle driving system also responded quickly to the requirements (Fig. 7(*e*)).

5.2 Right-Angle Turning Condition. The experimental results of tracking along an L right-angled road are shown in Fig. 8. Along this route (mapped in Fig. 8(*a*)), the maximum position and heading deviations were 79.7 cm and 0.47 rad, respectively (Fig. 8(*b*)). The RMSE of position and direction were 28.2931 cm and 0.0792 rad. The deviations were largest in at NOC = [500, 550], the early stage of turning the curve. At this

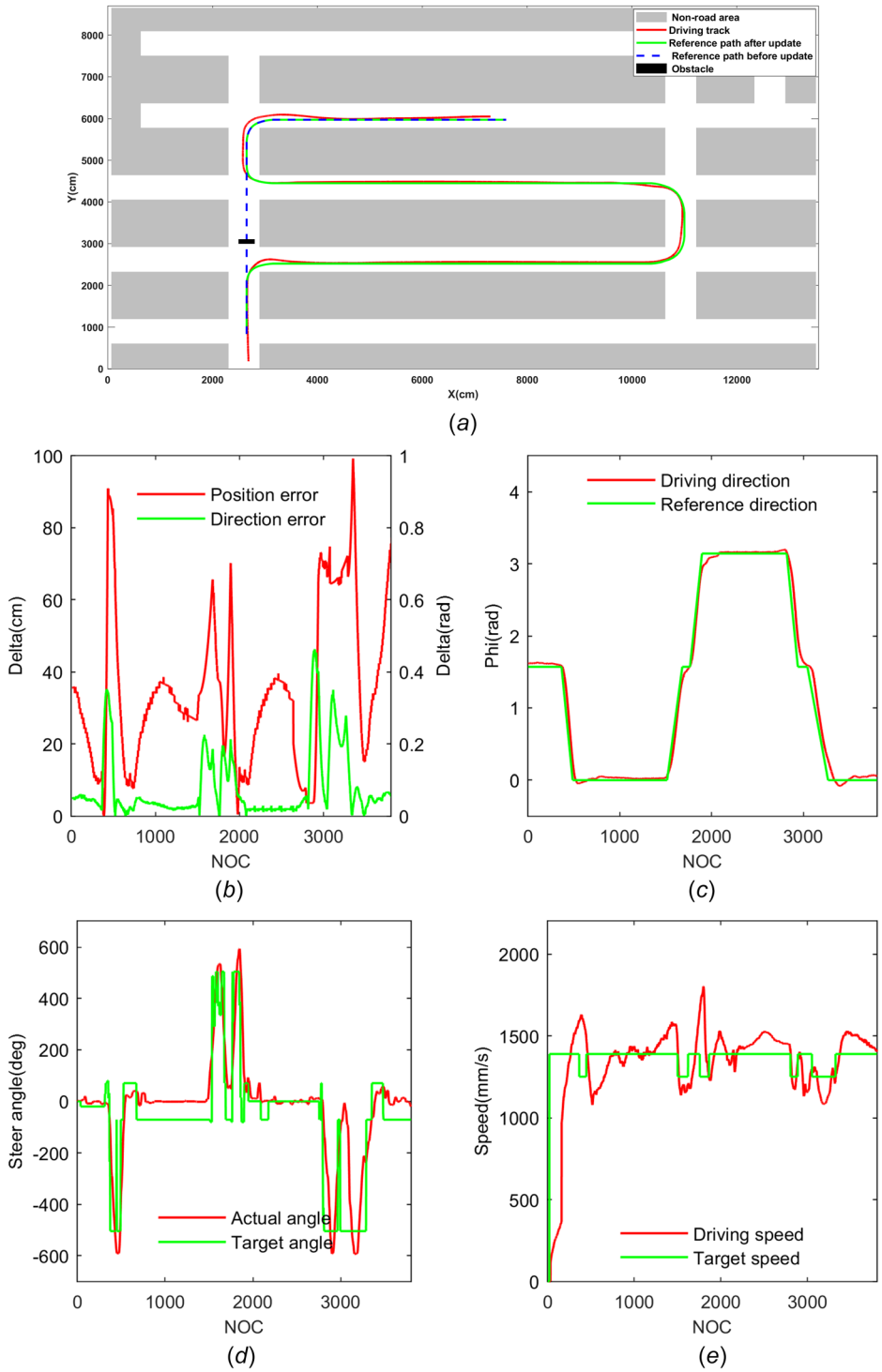

Fig. 10 Results of the obstacle avoidance experiment: (*a*) tracking-control effect chart, (*b*) vehicle position and heading error, (*c*) vehicle driving and reference directions, (*d*) vehicle steering-wheel target and execution angles, and (*e*) vehicle target and driving speeds

time, the steering mechanism of the vehicle has not yet performed the steering action, and there is a delay, so the error at this stage is high. The positional deviation was reduced as the heading was quickly adjusted after the controller working. The vehicle continued to follow the reference path. During the tracking process, the heading was in the straight-line tracking phase and the heading control was stable, but the control lagged in the steering phase, causing a large tracking deviation Fig. 8(*c*)). The steering-wheel angle was relatively smooth during the tracking process, but response lag was evident (Fig. 8(*d*)). Meanwhile, the vehicle resistance was increased by lateral deformation of the tire during the steering process, so the speed dropped in the NOC = [500,600] phase (Fig. 8(*e*)).

5.3 Rectangular Condition. Along the rectangular route (mapped in Fig. 9(*a*)), the maximum deviations in the position and heading directions were 88.6 cm and 0.58 rad, respectively (Fig. 9(*b*)). The RMSE of position and direction were 49.2486 cm and 1.4453 rad. The main reason is the same as that in Sec. 5.2.

The steering mechanism is lagging, resulting in a large deviation. The vehicle slightly deviated from its intended path during the steering and driving phases (Fig. 9(*c*)). During the NOC = [3400, 3700] phase, deviations were caused by insufficient angle execution of the steering wheel after steering. Under this working condition, the steering-wheel angle control of the vehicle was relatively stable (Fig. 9(*d*)), but in the NOC = [2800, 3100] and NOC = [3400, 3700] phases, the return execution during the steering-wheel return phase was insufficient, necessitating further optimization of the entire steering mechanism. The vehicle-driving system quickly responded to the speed requirements. In the NOC = [1000, 1500] phase, the downhill road environment induced a dramatic change in the vehicle speed and slight fluctuations in the driving system control (Fig. 9(*e*)).

5.4 Obstacle Avoidance Condition. Figure 10(*a*) shows an obstacle placed in the planned route. After encountering the obstacle and regenerating the reference path, the vehicle deviated by a maximum of 99.24 cm in position and 0.46 rad in direction (Fig. 10(*b*)). The RMSE of position and direction were 41.0903 cm and 0.1223 rad. Similarly, the vehicle has gone through five turning processes, and the lag of the steering mechanism causes a high deviation during the steering process, which is also one of the main reasons for the position error. The fluctuations differed in the three steering stages (NOC = [300, 500], NOC = [1500, 2000], NOC = [2800, 3000]). The heading and steering-wheel angle controls of the vehicle were relatively smooth during driving (Figs. 10(*c*) and 10(*d*)). In the current design, the execution signal of the steering wheel is the voltage change of the booster motor. The response time and execution accuracy of this signal must be improved to match the target angle given by the MPC. The vehicle speed quickly responded to the speed requirements, but during downhill driving and sections of increased steering friction, the adjustment process tended to oscillate (Fig. 10(*e*)).

6 Conclusion

This paper proposes a path-tracking algorithm based on the QPKWIK solver model, which is suitable for the path tracking of autonomous vehicles. Based on the Ackerman steering principle, a kinematic model for driving the car was first established. This model was linearized and discretized into the corresponding discrete state-space equation of the vehicle. Second, to guide the vehicle along the reference path generated by the path planning algorithm, a path-tracking controller that meets the vehicle model and driving requirements was designed through an MPC algorithm. Third, the problem of solving the objective function of the controller was transformed into a QP solution and was combined with the dual algorithm by inverting the Hessian matrix. Replacing the Hessian matrix with the L_{inv} matrix accelerated the iterative solution process. Finally, a real vehicle test was conducted in a closed parking lot. Experiments verified the superiority of the path-tracking control algorithm. In the present article, the solver was implemented by the quadratic programming method. In future work, we will consider other solving methods of the solver, such as further optimization of the QP solver. In addition, the use of other control methods is also one of the methods we consider, such as robust control.

Nomenclature

CG = the center of gravity of the vehicle
F_{yf} = lateral tire force on front tires
F_{yr} = lateral tire force on rear tires
L = total wheel base $(l_f + l_r)$
l_f = longitudinal distance from c.g. to front tires
l_r = longitudinal distance from c.g. to rear tires
m = total mass of the vehicle
M = the axes of the rear axles
N = the axes of the front axles
R = turning radius of the rear wheels
u = the control variable
W = width of the vehicle
X, Y = global axes
(X_f, Y_f) = the center coordinates of the front axles of the vehicle
(X_r, X_r) = the center coordinates of the rear axles of the vehicle
α = steering angle of outer wheels
β = steering angle of inner wheels
δ = steering wheel angle
$\dot{\xi}$ = the vehicle state space
v_f = speed of the center front axles of the vehicle
v_r = speed of the center rear axles of the vehicle
φ = yaw angle of the vehicle body in global axes
ϕ_{steer} = the vehicle steering-wheel angle
ω = the vehicle yaw rate

Appendix A: The Coordinates of the Vehicle Corner Point

The coordinates of points of A, B, C, and D can be calculated as follows:

Point A (X_A, Y_A) at the front-left corner of the vehicle

$$\begin{cases} X_A = X_r + \cos\left(\varphi + \arctan\left(\dfrac{W}{2(L+l_f)}\right)\right) * \sqrt{\left(\dfrac{W}{2}\right)^2 + (L+l_f)^2} \\ Y_A = Y_r + \sin\left(\varphi + \arctan\left(\dfrac{W}{2(L+l_f)}\right)\right) * \sqrt{\left(\dfrac{W}{2}\right)^2 + (L+l_f)^2} \end{cases} \tag{A1}$$

Point B (X_B, Y_B) at the front-right corner of the vehicle

$$\begin{cases} X_B = X_r + \cos\left(\varphi - \arctan\left(\dfrac{W}{2(L+l_f)}\right)\right) * \sqrt{\left(\dfrac{W}{2}\right)^2 + (L+l_f)^2} \\ Y_B = Y_r + \sin\left(\varphi - \arctan\left(\dfrac{W}{2(L+l_f)}\right)\right) * \sqrt{\left(\dfrac{W}{2}\right)^2 + (L+l_f)^2} \end{cases} \tag{A2}$$

Point C (X_C, Y_C) at the right-rear corner of the vehicle

$$\begin{cases} X_C = X_r + \sin\left(\varphi - \arctan\left(\dfrac{2l_r}{W}\right)\right) * \sqrt{\left(\dfrac{W}{2}\right)^2 + {l_r}^2} \\ Y_C = Y_r - \cos\left(\varphi - \arctan\left(\dfrac{2l_r}{W}\right)\right) * \sqrt{\left(\dfrac{W}{2}\right)^2 + {l_r}^2} \end{cases} \tag{A3}$$

Point D (X_D, Y_D) at the rear-left corner of the vehicle

$$\begin{cases} X_D = X_r - \left(\sin\varphi + \arctan\left(\dfrac{2l_r}{W}\right)\right) * \sqrt{\left(\dfrac{W}{2}\right)^2 + {l_r}^2} \\ Y_D = Y_r + \left(\cos\varphi + \arctan\left(\dfrac{2l_r}{W}\right)\right) * \sqrt{\left(\dfrac{W}{2}\right)^2 + {l_r}^2} \end{cases} \tag{A4}$$

Appendix B: Taylor Expansion of Eq. (16)

Expanding Eq. (12) as a Taylor series around the path. Reference point, we get

$$\dot{\boldsymbol{\xi}}_{\mathbf{r}} = f(\boldsymbol{\xi}_{\mathbf{r}}, \mathbf{u}_{\mathbf{r}}) + \frac{\partial f(\xi,u)}{\partial \xi}\Big|_{u=u_r}^{\xi=\xi_r}(\xi-\xi_r) + \frac{\partial f(\xi,u)}{\partial u}\Big|_{u=u_r}^{\xi=\xi_r}(u-u_r) + \frac{1}{2!}\times\frac{\partial^2 f(\xi,u)}{\partial \xi^2}\Big|_{u=u_r}^{\xi=\xi_r}(\xi-\xi_r)^2 + \frac{1}{2!}\times\frac{\partial^2 f(\xi,u)}{\partial u^2}\Big|_{u=u_r}^{\xi=\xi_r}(u-u_r)^2 + \cdots + \frac{1}{n!}\times\frac{\partial^n f(\xi,u)}{\partial \xi^n}\Big|_{u=u_r}^{\xi=\xi_r}(\xi-\xi_r)^n + \frac{1}{n!}\times\frac{\partial^n f(\xi,u)}{\partial u^n}\Big|_{u=u_r}^{\xi=\xi_r}(u-u_r)^n + R_n(\xi_r,u_r) \tag{B1}$$

Ignoring the higher-order terms, the following can be obtained:

$$\dot{\boldsymbol{\xi}}_{\mathbf{r}} = f(\boldsymbol{\xi}_{\boldsymbol{r}}, \mathbf{u}_{\mathbf{r}}) + \frac{\partial f(\xi,u)}{\partial \xi}\Big|_{u=u_r}^{\xi=\xi_r}(\xi-\xi_r) + \frac{\partial f(\xi,u)}{\partial u}\Big|_{u=u_r}^{\xi=\xi_r}(u-u_r) \tag{B2}$$

From Eq. (12) and Eq. (B2), $\dot{\tilde{\xi}}$ is obtained as

$$\dot{\tilde{\boldsymbol{\xi}}}_{\mathbf{r}} = \begin{bmatrix} \dot{X}-\dot{X}_r \\ \dot{Y}-\dot{Y}_r \\ \dot{\varphi}-\dot{\varphi}_r \end{bmatrix} = \begin{bmatrix} 0 & 0 & -v_r\sin\varphi_r \\ 0 & 0 & v_r\cos\varphi_r \\ 0 & 0 & 0 \end{bmatrix}\begin{bmatrix} X-X_r \\ Y-Y_r \\ \varphi-\varphi_r \end{bmatrix} + \begin{bmatrix} \cos\varphi_r & 0 \\ \sin\varphi_r & 0 \\ \dfrac{\tan\delta_r}{l} & \dfrac{v_r}{l\cos^2\delta_r} \end{bmatrix}\begin{bmatrix} v-v_r \\ \delta-\delta_r \end{bmatrix} \tag{B3}$$

Turning it into a general form, we get

$$\dot{\tilde{\boldsymbol{\xi}}}_{\mathbf{r}} = \begin{bmatrix} \dot{x}-\dot{x}_r \\ \dot{y}-\dot{y}_r \\ \dot{\varphi}-\dot{\varphi}_r \end{bmatrix} = \begin{bmatrix} 0 & 0 & -v_r\sin\varphi_r \\ 0 & 0 & v_r\cos\varphi_r \\ 0 & 0 & 0 \end{bmatrix}\begin{bmatrix} x-x_r \\ y-y_r \\ \varphi-\varphi_r \end{bmatrix} + \begin{bmatrix} \cos\varphi_r & 0 \\ \sin\varphi_r & 0 \\ \dfrac{\tan\delta_r}{l} & \dfrac{v_r}{l\cos^2\delta_r} \end{bmatrix}\begin{bmatrix} v-v_r \\ \delta-\delta_r \end{bmatrix} \tag{B4}$$

Appendix C: State Space Calculation

By Substituting Eq. (18) into Eq. (17), the Space State $\boldsymbol{\psi}(\mathbf{k})$ is shown as follows:

$$\begin{aligned}
\boldsymbol{\psi}(1) &= \tilde{\mathbf{A}}_t\psi(0) + \tilde{\mathbf{B}}_t\Delta\boldsymbol{u}(0) \\
\boldsymbol{\psi}(2) &= \tilde{\mathbf{A}}_t\boldsymbol{\psi}(1) + \tilde{\mathbf{B}}_t\Delta\mathbf{u}(1) = \tilde{\mathbf{A}}_t\big(\tilde{\mathbf{A}}_t\boldsymbol{\psi}(0) + \tilde{\mathbf{B}}_t\Delta\mathbf{u}(0)\big) + \tilde{\mathbf{B}}_t\Delta\mathbf{u}(1) \\
&= \tilde{\mathbf{A}}_t^{\,2}\boldsymbol{\psi}(0) + \tilde{\mathbf{A}}_t\tilde{\mathbf{B}}_t\boldsymbol{\psi}(0) + \tilde{\mathbf{B}}_t\Delta\mathbf{u}(1) \\
\boldsymbol{\psi}(3) &= \tilde{\mathbf{A}}_t\boldsymbol{\psi}(2) + \tilde{\mathbf{B}}_t\Delta\mathbf{u}(2) = \tilde{\mathbf{A}}_t^{\,3}\boldsymbol{\psi}(0) + \tilde{\mathbf{A}}_t^{\,2}\tilde{\mathbf{B}}_t\boldsymbol{\psi}(0) \\
&\quad + \tilde{\mathbf{A}}_t\tilde{\mathbf{B}}_t\Delta\mathbf{u}(1) + \tilde{\mathbf{B}}_t\Delta\mathbf{u}(2) \\
&\quad \cdots\cdots \\
\boldsymbol{\psi}(k) &= \tilde{\mathbf{A}}_t^{\,k}\boldsymbol{\psi}(0) + \tilde{\mathbf{A}}_t^{\,k-1}\tilde{\mathbf{B}}_t\Delta\mathbf{u}(0) + \tilde{\mathbf{A}}_t^{\,k-2}\tilde{\mathbf{B}}_t\Delta\boldsymbol{u}(1) \\
&\quad + \cdots + \tilde{\mathbf{A}}_t\tilde{\mathbf{B}}_t\Delta\mathbf{u}(k-2) + \tilde{\mathbf{B}}_t\Delta\mathbf{u}(k-1) \\
&= \tilde{\mathbf{A}}_t^{\,k}\boldsymbol{\psi}\Big(0) + \sum_{j=0}^{k-1}\Big(\tilde{A}_t^{\,k-1-j}\tilde{B}_t\Delta u(j)\Big)
\end{aligned} \tag{C1}$$

For the control variable $\boldsymbol{\eta}(k)$

$$\begin{aligned}
\boldsymbol{\eta}(1) &= \tilde{\boldsymbol{C}}_{k,t}\boldsymbol{\psi}(1) = \tilde{\boldsymbol{A}}_t\boldsymbol{\psi}(0) + \tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(0) \\
\boldsymbol{\eta}(2) &= \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{A}}_t^{\,k}\boldsymbol{\psi}(1) + \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(1) = \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{A}}_t^{\,k}2\boldsymbol{\psi}(0) \\
&\quad + \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{A}}_t^{\,k}\tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(0) + \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(1) \\
\boldsymbol{\eta}(3) &= \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{A}}_t^{\,k}\boldsymbol{\psi}(2) + \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(2) \\
&= \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{A}}_t^{\,k}3\boldsymbol{\psi}(0) + \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{A}}_t^{\,k}2\tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(0) \\
&\quad + \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{A}}_t^{\,k}\tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(1) + \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{A}}_t^{\,k}\tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(2) \\
&\quad \cdots\cdots \\
\boldsymbol{\eta}(k) &= \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{A}}_t^{\,k}\boldsymbol{\psi}(k+j-1) + \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(\mathrm{k}+\mathrm{j}-1) \\
&= \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{A}}_t^{\,k}\boldsymbol{j}\boldsymbol{\psi}(0) + \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{A}}_t^{\,k}\boldsymbol{j} - 1\tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(0) \\
&\quad + \cdots + \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(\mathrm{k}+\mathrm{j}-1) \\
&= \tilde{\boldsymbol{C}}_{k,t}\boldsymbol{\psi}(\mathrm{k}) + \tilde{\boldsymbol{A}}_t^{\,k}\boldsymbol{\psi}(0) + \tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(\mathbf{0}) \\
&= \tilde{\boldsymbol{C}}_{k,t}\left(\tilde{\boldsymbol{A}}_t^{\,k}\boldsymbol{\psi}(0) + \sum_{j=0}^{k-1}\left(\tilde{\boldsymbol{A}}_t^{\,k-1-j}\tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(\boldsymbol{j})\right)\right) \\
&= \tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{A}}_t^{\,k}\boldsymbol{\psi}(0) + \sum_{j=0}^{k-1}\tilde{\boldsymbol{C}}_{k,t}\tilde{\boldsymbol{A}}_t^{\,k-1-j}\tilde{\boldsymbol{B}}_t\boldsymbol{\Delta u}(\boldsymbol{j})
\end{aligned} \tag{C2}$$

References

[1] Gordon, T. J., and Lidberg, M., 2015, "Automated Driving and Autonomous Functions on Road Vehicles," Veh. Sys. Dyn., **53**(7), pp. 958–994.
[2] Badue, C., Guidolini, R., Carneiro, R. V., Azevedo, P., Cardoso, V. B., Forechi, A., Jesus, L., Berriel, R., Paixão, T. M., Mutz, F., de Paula Veronese, L., Oliveira-Santos, T., and De Souza, A. F., 2021, "Self-Driving Cars: A Survey," Expert. Syst. Appl., **165** p. 113816.
[3] Veres, S. M., Molnar, L., Lincoln, N. K., and Morice, C. P., 2011, "Autonomous Vehicle Control Systems—a Review of Decision Making," Proc. Inst. Mech. Eng. Part I-J. Syst. Control Eng., **225**(2), pp. 155–195.
[4] Kuutti, S., Bowden, R., Jin, Y., Barber, P., and Fallah, S., 2021, "A Survey of Deep Learning Applications to Autonomous Vehicle Control," IEEE Trans. Intell. Transp. Syst., **22**(2), pp. 712–733.
[5] Wang, H., Liu, B., Ping, X., and An, Q., 2019, "Path Tracking Control for Autonomous Vehicles Based on an Improved MPC," IEEE Access, **7**, pp. 161064–161073.
[6] Paden, B., Čáp, M., Yong, S. Z., Yershov, D., and Frazzoli, E., 2016, "A Survey of Motion Planning and Control Techniques for Self-Driving Urban Vehicles," IEEE Trans. Intell. Veh., **1**(1), pp. 33–55.
[7] Zheng, Y., and Özgüner, Ü., 2007, "A Composite Model for Vehicle Formation and Path Selection on a Cellular Structured Map," ASME J. Dyn. Sys., Meas., Control., **129**(5), pp. 644–653.
[8] Likhachev, M., and Ferguson, D., 2009, "Planning Long Dynamically Feasible Maneuvers for Autonomous Vehicles," Int. J. Robot. Res., **28**(8), pp. 933–945.
[9] Karaman, S., and Frazzoli, E., 2011, "Sampling-Based Algorithms for Optimal Motion Planning," Int. J. Robot. Res., **30**(7), pp. 846–894.
[10] Brezak, M., and Petrovi, I., 2014, "Real-Time Approximation of Clothoids With Bounded Error for Path Planning Applications," IEEE Trans. Robot., **30**(2), pp. 507–515.

[11] Funke, J., and Christian Gerdes, J., 2015, "Simple Clothoid Lane Change Trajectories for Automated Vehicles Incorporating Friction Constraints," ASME J. Dyn. Syst., Meas., Control, **138**(2), p. 021002.
[12] Norouzi, A., Kazemi, R., and Abbassi, O. R., 2019, "Path Planning and Re-Planning of Lane Change Manoeuvres in Dynamic Traffic Environments," Int. J. Veh. Auton. Syst., **14**(3), pp. 239–264.
[13] Chen, Y., Cai, Y., Zheng, J., and Thalmann, D., 2017, "Accurate and Efficient Approximation of Clothoids Using Bezier Curves for Path Planning," IEEE Trans. Robot., **33**(5), pp. 1242–1247.
[14] Berglund, T., Brodnik, A., Jonsson, H., Staffanson, M., and Soderkvist, I., 2010, "Planning Smooth and Obstacle-Avoiding B-Spline Paths for Autonomous Mining Vehicles," IEEE Trans. Autom. Sci. Eng., **7**(1), pp. 167–172.
[15] Elbanhawi, M., Simic, M., and Jazar, R., 2018, "Receding Horizon Lateral Vehicle Control for Pure Pursuit Path Tracking," J. Vib. Control., **24**(3), pp. 619–642.
[16] Amer, N. H., Zamzuri, H., Hudha, K., and Kadir, Z. A., 2017, "Modelling and Control Strategies in Path Tracking Control for Autonomous Ground Vehicles: A Review of State of the Art and Challenges," J. Intell. Robot. Syst., **86**(2), pp. 225–254.
[17] Campbell, S. F., 2007, "Steering Control of an Autonomous Ground Vehicle With Application to the DARPA Urban Challenge," Ph.D. thesis, Massachusetts Institute of Technology, Cambridge, MA.
[18] Törő, O., Bécsi, T., and Aradi, S., 2016, "Design of Lane Keeping Algorithm of Autonomous Vehicle," Period. Polytech. Transp. Eng., **44**(1), pp. 60–68.
[19] Fan, Z., and Chen, H., 2016, "Study on Path Following Control Method for Automatic Parking System Based on LQR," SAE Int. J. Passenger Cars Electron. Electr. Syst., **10**(1), pp. 41–49.
[20] Cordeiro, R. A., Azinheira, J. R., de Paiva, E. C., and Bueno, S. S., 2013, "Dynamic Modeling and Bio-Inspired LQR Approach for Off-Road Robotic Vehicle Path Tracking," 16th International Conference Advanced Robotics (ICAR), Montevideo, Uruguay, Nov. 25–29, pp. 1–6.
[21] Solea, R., and Nunes, U., 2007, "Trajectory Planning and Sliding-Mode Control Based Trajectory-Tracking for Cybercars," Integr. Comput.-Aided Eng., **14**(1), pp. 33–47.
[22] Wang, J., Steiber, J., and Surampudi, B., 2009, "Autonomous Ground Vehicle Control System for High-Speed and Safe Operation," Int. J. Veh. Auton. Syst., **7**(1/2), pp. 18–35.
[23] Hu, C., Jing, H., Wang, R., Yan, F., and Chadli, M., 2016, "Robust H∞ Output-Feedback Control for Path Following of Autonomous Ground Vehicles," Mech. Syst. Signal Proc., **70-71**, pp. 414–427.
[24] Chen, I. M., and Chan, C.-Y., 2021, "Deep Reinforcement Learning Based Path Tracking Controller for Autonomous Vehicle," Proc. Inst. Mech. Eng. Part D-J. Autom. Eng., **235**(2–3), pp. 541–551.
[25] Shan, Y., Zheng, B., Chen, L., Chen, L., and Chen, D., 2020, "A Reinforcement Learning-Based Adaptive Path Tracking Approach for Autonomous Driving," IEEE Trans. Veh. Technol., **69**(10), pp. 10581–10595.
[26] Brown, M., Funke, J., Erlien, S., and Gerdes, J. C., 2017, "Safe Driving Envelopes for Path Tracking in Autonomous Vehicles," Control Eng. Pract., **61**, pp. 307–316.
[27] Jalali, M., Khajepour, A., Chen, S.-K., and Litkouhi, B., 2017, "Handling Delays in Yaw Rate Control of Electric Vehicles Using Model Predictive Control With Experimental Verification," ASME J. Dyn. Syst., Meas., Control, **139**(12), p. 121001.
[28] Wang, Z., Bai, Y., Wang, J., and Wang, X., 2019, "Vehicle Path-Tracking Linear-Time-Varying Model Predictive Control Controller Parameter Selection Considering Central Process Unit Computational Load," ASME J. Dyn. Syst., Meas., Control., **141**(5), p. 051004.
[29] Mayne, D. Q., Rawlings, J. B., Rao, C. V., and Scokaert, P. O. M., 2000, "Constrained Model Predictive Control: Stability and Optimality," Automatica, **36**(6), pp. 789–814.
[30] Sun, C., Zhang, X., Zhou, Q., and Tian, Y., 2019, "A Model Predictive Controller With Switched Tracking Error for Autonomous Vehicle Path Tracking," IEEE Access, **7**, pp. 53103–53114.
[31] Guo, H., Cao, D., Chen, H., Sun, Z., and Hu, Y., 2019, "Model Predictive Path Following Control for Autonomous Cars Considering a Measurable Disturbance: Implementation, Testing, and Verification," Mech. Syst. Signal Proc., **118**, pp. 41–60.
[32] Vasantharajan, S., Viswanathan, J., and Biegler, L. T., 1990, "Reduced Successive Quadratic Programming Implementation for Large-Scale Optimization Problems With Smaller Degrees of Freedom," Comput. Chem. Eng., **14**(8), pp. 907–915.
[33] Falcone, P., Eric Tseng, H., Borrelli, F., Asgari, J., and Hrovat, D., 2008, "MPC-Based Yaw and Lateral Stabilisation Via Active Front Steering and Braking," Veh. Syst. Dyn., **46**(suppl. 1), pp. 611–628.
[34] Goldfarb, D., and Idnani, A., 1983, "A Numerically Stable Dual Method for Solving Strictly Convex Quadratic Programs," Math. Program., **27**(1), pp. 1–33.
[35] Schmid, C., and Biegler, L. T., 1994, "Reduced Hessian Successive Quadratic Programming for Realtime Optimization," Adv. Control Chem. Process., **27**(2), pp. 173–178.
[36] Rajamani, R., 2011, *Vehicle Dynamics and Control*, Springer Science & Business Media, Berlin, Germany.
[37] Shekh, M., Umrao, O., and Singh, D., 2020, "Kinematic Analysis of Steering Mechanism: A Review," Proceedings of International Conference in Mechanical and Energy Technology, Greater Noida, India, Nov. 7–8, pp. 529–540.
[38] Norouzi, A., Masoumi, M., Barari, A., and Farrokhpour Sani, S., 2019, "Lateral Control of an Autonomous Vehicle Using Integrated Backstepping and Sliding Mode Controller," Proc. Inst. Mech. Eng. Part K-J. Multi-Body Dyn., **233**(1), pp. 141–151.
[39] Pacejka, H. B., and Bakker, E., 1992, "The Magic Formula Tyre Model," Veh. Syst. Dyn., **21**(suppl. 1), pp. 1–18.
[40] Kapania, N. R., and Gerdes, J. C., 2015, "Design of a Feedback-Feedforward Steering Controller for Accurate Path Tracking and Stability at the Limits of Handling," Veh. Syst. Dyn., **53**(12), pp. 1687–1704.
[41] Cannon, M., Liao, W., and Kouvaritakis, B., 2008, "Efficient MPC Optimization Using Pontryagin's Minimum Principle," Int. J. Robust Nonlinear Control, **18**(8), pp. 831–844.
[42] Maciejowski, J. M., 2002, *Predictive Control: With Constraints*, Pearson Education, Englewood Cliffs, NJ.
[43] Wang, L., 2009, *Model Predictive Control System Design and Implementation Using MATLAB®*, Springer Science & Business Media, Berlin, Germany.
[44] Rokonuzzaman, M., Mohajer, N., Nahavandi, S., and Mohamed, S., 2021, "Review and Performance Evaluation of Path Tracking Controllers of Autonomous Vehicles," IET Intell. Transp. Syst., **15**(5), pp. 646–670.
[45] Schmid, C., and Biegler, L. T., 1994, "Quadratic Programming Methods for Reduced Hessian SQP," Comput. Chem. Eng., **18**(9), pp. 817–832.
[46] Ternet, D. J., and Biegler, L. T., 1998, "Recent Improvements to a Multiplier-Free Reduced Hessian Successive Quadratic Programming Algorithm," Comput. Chem. Eng., **22**(7–8), pp. 963–978.